LAB-04 NOTES – UVM FACTORY OVERRIDES (FULL DETAILED VERSION)

1. Goal / Objective

The objective of Lab-04 was to understand Factory Overrides in UVM.

In previous labs, every packet created through:
yapp_packet::type_id::create()

always produced a yapp_packet object.

Lab-04 teaches how to replace one object type with another without modifying existing sequences, drivers, or reusable code.

Main concepts learned:
• Factory Override
• Type Override
• Object Substitution
• Inheritance
• Reusability
• Factory-Based Design

2. What We Had Before Starting Lab-04

A complete UVM hierarchy:
top -> test -> env -> agent -> driver & sequencer

All packet creation resulted in yapp_packet objects.
Factory had no overrides configured.

3. Problem Statement

Suppose we need shorter packets.

Without Factory Override:
• Modify sequences
• Modify drivers
• Modify tests

This reduces reusability.

UVM solution:
• Create a new packet type
• Tell Factory to use it
• Keep existing code unchanged

4. Coding Flow Followed

1. Create Derived Packet
2. Register New Packet

3. Add New Constraints
4. Create Override Test
5. Apply Factory Override
6. Run Existing Sequences
7. Verify Override

5. Step-by-Step Development

Step 1: Create Derived Packet

class short_yapp_packet extends yapp_packet;

Why?
Need a modified packet without touching original packet code.

Learning:
Inheritance extends existing functionality.

Step 2: Register Derived Packet

`uvm_object_utils(short_yapp_packet)

Why?
Factory can only create registered classes.

Learning:
Every factory-created object must be registered.

Step 3: Add New Constraints

constraint c_short {
length_i inside {[1:15]};
}

Why?
Need shorter packets than original yapp_packet.

Learning:
Derived classes can modify behavior using constraints.

Step 4: Create Override Test

class short_packet_test extends base_test;

Why?
Overrides should be controlled from test level.

Learning:
Tests configure the environment.

Step 5: Apply Factory Override

```
yapp_packet::type_id::set_type_override(
short_yapp_packet::get_type()
);
```

Meaning:
Whenever Factory sees yapp_packet::create(),
it creates short_yapp_packet instead.

Learning:
Factory can substitute object types automatically.

Step 6: Keep Existing Sequence Unchanged

```
pkt = yapp_packet::type_id::create("pkt");
```

No changes required.

Learning:
Reusable code remains untouched.

Step 7: Verify Override

Use:
pkt.print();
or
pkt.sprint();

Verify object type is short_yapp_packet.

Learning:
Override is transparent to sequences and drivers.

6. Internal Working of Factory Override

Before Override:

Sequence
|
Factory
|
yapp_packet

After Override:

Sequence
|

Factory
|
short_yapp_packet

Sequence never knows the difference.
Factory handles substitution.

7. Testbench Evolution

Before Lab-04:
base_test
yapp_packet

After Lab-04:
base_test
short_packet_test
yapp_packet
short_yapp_packet

New packet variants can now be introduced without modifying reusable code.

8. Why Not Use new()?

Wrong:
pkt = new();

Factory cannot intercept object creation.

Correct:
pkt = yapp_packet::type_id::create("pkt");

Factory can perform overrides.

Learning:
Factory overrides only work through create().

9. Issues Faced and Solutions

Issue 1:
Override not working.

Cause:
Used new() instead of create().

Fix:
Always use type_id::create().

Issue 2:
Wrong registration macro.

Cause:
Used `uvm_component_utils for packet class.

Fix:
Use `uvm_object_utils.

Issue 3:
Override applied too late.

Cause:
Override placed after object creation.

Fix:
Apply override inside build_phase().

Issue 4:
Confusion between variable type and actual object type.

Learning:
Variable type may be yapp_packet while actual object is short_yapp_packet due to factory override.

10. Real Project Importance

Factory Overrides are heavily used in industry.

Examples:
• Short Packet
• Long Packet
• Error Packet
• Corrupted Packet
• Protocol Variant Packet

All can be substituted without modifying reusable sequences.

11. Interview Questions

Q1. Why use Factory Override?

To modify behavior without modifying reusable code.

Q2. Difference between new() and create()?

new() = Direct creation
create() = Factory creation

Q3. When does override work?

Only when object is created through factory.

Q4. What is set_type_override()?

Replaces one object type with another.

Q5. Why is Factory important?

Improves flexibility and reuse.

12. Final Learning Summary

Factory = Object Creator
Override = Object Replacement
Inheritance = Extension
create() = Factory Creation
new() = Direct Creation
Test = Controls Override

13. One-Line Revision

Lab-04 introduced Factory Overrides, allowing one packet type to be replaced with another without modifying existing sequence or driver code, greatly improving reuse and flexibility.

Memory Trick

Lab-01 = Build the Packet
Lab-02 = Build the Roads
Lab-03 = Build the Controller
Lab-04 = Change the Packet Type
Lab-05 = Generate Traffic

Think of ordering a Pizza and receiving a Veg Pizza without changing the order form.
That is exactly how Factory Override works.